Part 01: Questions

*** Question 1: What is the difference between int.Parse and Convert.ToInt32 when handling null inputs?**

* Answer:

* int.Parse(null) throws an ArgumentNullException.

* Convert.ToInt32(null) handles null safely and returns 0.

*** Question 2: Why is TryParse recommended over Parse in user-facing applications?**

* Answer: TryParse does not throw runtime exceptions when parsing fails. Instead, it returns false, which provides better performance and avoids application crashes during bad user inputs.

*** Question 3: Explain the real purpose of the GetHashCode() method.**

* Answer: The primary purpose of GetHashCode() is to return an integer value used to quickly identify and locate objects within hash-based data structures like Dictionary and HashSet.

*** Question 4: What is the significance of reference equality in .NET?**

* Answer: Reference equality means two or more variables point to the exact same memory location on the heap. Any modification made through one reference variable immediately affects the object for all other references pointing to it.

*** Question 5: Why is string immutable in C#?**

* Answer: Strings are made immutable to guarantee thread safety, security, string interning (reusing identical string literals in memory), and hash code stability.

*** Question 6: How does StringBuilder address the inefficiencies of string concatenation?**

* Answer: StringBuilder operates on an internal dynamic mutable buffer. Modifying text inside a StringBuilder updates memory in-place without creating new objects on the heap every time.

*** Question 7: Why is StringBuilder faster for large-scale string modifications?**

* Answer: Because it avoids continuous memory reallocations and reduces Garbage Collection (GC) pressure by using a pre-allocated internal character buffer.

*** Question 8: Which string formatting method is most used and why?**

* Answer: String Interpolation ($"...") is the most widely used because it offers the cleanest, most readable syntax and allows embedding expressions directly inside the text.

*** Question 9: Explain how StringBuilder is designed to handle frequent modifications compared to strings.**

* Answer: StringBuilder maintains an expandable character array (char[]). When performing operations like Append, Insert, or Remove, it directly manipulates the existing array instead of constructing a completely new object like standard string does.

Part 02: Conceptual Questions

***2 What's Enum data type, when is it used? And name three common built-in enums used frequently?**

* Definition: An enum (enumeration) is a value type containing a set of named integer constants.

* When to use: Used when a variable can only take one out of a fixed set of predefined options (e.g., days of the week, user roles, order statuses).

* Three Common Built-in Enums:

* DayOfWeek

* ConsoleColor

* TypeCode

*** 3 What are scenarios to use string vs StringBuilder?**

* Use string: For small/few modifications, fixed literals, read-only text, or simple concatenations.

* Use StringBuilder: For heavy string manipulations, dynamic text building inside loops, or building large documents/strings at runtime.

Part 03: Bonus Questions

*** Self study report:**

* Overview: Self-study topics generally focus on deep-dive concepts like Garbage Collection (GC) mechanics, Value Types vs Reference Types (Stack vs Heap allocation), and memory management optimization in C#.

*** 2 What is meant by user-defined constructor and its role in initialization?**

* Answer: A user-defined constructor is a custom method inside a class created manually to enforce mandatory initial values and run setup code when an instance (object) is instantiated via the new keyword.

*** 3 Compare between Array and Linked List:**

1. Memory Allocation

Array: Allocates memory in contiguous (sequential) memory blocks.

Linked List: Allocates memory in non-contiguous memory locations using nodes connected via pointers.

2. Access Time

Array: Fast O(1) access time because elements can be accessed directly using their index.

Linked List: Slow O(n) access time because it must traverse sequentially through nodes from the head to reach an element.

3. Insertion & Deletion

Array: Slow O(n) operation because adding or removing elements requires shifting subsequent elements in memory.

Linked List: Fast O(1) operation when inserting or deleting nodes if the position pointer is known, as it only updates reference pointers.

4. Size & Flexibility

Array: Fixed size defined at creation time that cannot be easily resized.

Linked List: Dynamic size that can grow or shrink automatically during runtime based on memory availability.